

Universidade Federal de Santa Catarina
Departamento de Informática e Estatística
Modelagem e Simulação

Desenvolvimento de Componente de Análise de Dados no GenESyS

Tema 11.3.1 – Foco em Diagnósticos Adicionais e Ajustes de Curvas

Autor: Gabriel Salmoria

Florianópolis, 2026

Sumário

1	Introdução	2
1.1	Contextualização	2
1.2	Objetivo	2
1.3	Justificativa	2
2	Desenvolvimento	4
2.1	Visão geral da arquitetura	4
2.2	Extensão da interface Fitter_if (M1)	4
2.3	Interface DataAnalyzer_if e structs de saída (M2)	5
2.4	Implementação: carregamento de dados e estatísticas descritivas (M3) . . .	5
2.5	Ajuste de distribuições (M4)	6
2.6	Testes de aderência (M5 e M6)	6
2.7	Análise de séries temporais (M7)	6
2.8	Inferência paramétrica (M8)	6
2.9	Integração com o GenESyS	7
3	Validação	8
3.1	Estratégia de teste	8
3.2	Seção 1 – Carregamento de dados	8
3.3	Seção 2 – Estatísticas descritivas	8
3.4	Seção 3 – Histograma e boxplot	8
3.5	Seção 4 – Ajuste de distribuições	8
3.6	Seção 5 – Testes de aderência	9
3.7	Seção 6 – Séries temporais	9
3.8	Seção 7 – Inferência paramétrica	9
3.9	Resultado da execução	9
3.10	Problemas encontrados e corrigidos	10
4	Conclusão	11

1 Introdução

1.1 Contextualização

O GenESyS (*Generic Extensible Simulator*) é uma plataforma de modelagem e simulação de eventos discretos desenvolvida em C++, cujo código-fonte é mantido em repositório público no GitHub [2]. A plataforma adota uma arquitetura extensível baseada em interfaces abstratas e implementações concretas intercambiáveis, permitindo a adição de novos componentes sem alteração do núcleo do simulador.

No contexto de simulação, a análise estatística dos resultados produzidos por modelos é uma etapa fundamental do processo científico. Após a execução de um modelo, é necessário verificar a distribuição dos dados de saída, ajustar distribuições teóricas, aplicar testes de aderência e realizar inferências sobre as populações simuladas [1]. Até o início deste trabalho, o pacote `source/tools/` do GenESyS já oferecia implementações para ajuste de distribuições (`Fitter_if`), testes de hipóteses (`HypothesisTester_if`) e resolução numérica (`Solver_if`), porém sem uma fachada unificada que integrasse todas essas capacidades em um único ponto de acesso.

1.2 Objetivo

O objetivo deste trabalho é implementar o componente `DataAnalyzer`, uma ferramenta de análise de dados integrada ao GenESyS que oferece:

- carregamento de dados a partir de memória ou de arquivos texto nos formatos suportados pelo simulador;
- estatísticas descritivas completas (média, mediana, moda, quartis, assimetria, curtose, entre outras);
- estruturas exploratórias de histograma e boxplot;
- ajuste de sete distribuições de probabilidade com seleção automática da melhor;
- testes de aderência qui-quadrado e Kolmogorov-Smirnov;
- análise de séries temporais (média móvel e autocorrelação);
- intervalos de confiança e testes de hipóteses para uma e duas populações.

O tema enquadra-se na diretriz 11.3.1 da proposta – *foco maior em diagnósticos adicionais e ajustes de curvas* – e é referenciado no GenESyS como `DataAnalyzer`.

1.3 Justificativa

A ausência de uma interface unificada de análise forçava o usuário a instanciar e coordenar manualmente `Fitter_if`, `HypothesisTester_if` e parsers de arquivo, além de reimplementar cálculos estatísticos básicos em cada contexto de uso. Um componente façade reduz o acoplamento entre camadas do simulador e serve como base para futuros painéis

gráficos de análise de resultados.

Do ponto de vista pedagógico, o trabalho exigiu compreensão profunda da arquitetura do GenESyS – especialmente o padrão `TraitsTools<T>` de injeção de implementações – e a integração de métodos numéricos já existentes na plataforma (integração por quadratura, busca de raízes por bissecção) para suportar novas funcionalidades.

2 Desenvolvimento

2.1 Visão geral da arquitetura

O componente foi desenvolvido em três camadas, seguindo o padrão arquitetural já adotado pelo GenESyS:

1. **Interface abstrata** (`DataAnalyzer_if.h`) – define os tipos de saída e os métodos puros virtuais;
2. **Implementação concreta** (`DataAnalyzerDefaultImpl1`) – realiza todos os cálculos e delega para `FitterDefaultImpl` e `HypothesisTesterDefaultImpl1`;
3. **Registro de traits** (`TraitsTools<DataAnalyzer_if>`) – vincula a interface à implementação concreta, tornando a substituição transparente.

A Tabela 1 lista os arquivos criados ou modificados.

Tabela 1: Arquivos modificados ou criados

Arquivo	Ação
<code>source/tools/Fitter_if.h</code>	Adicionado <code>setDataValues()</code>
<code>source/tools/FitterDefaultImpl.h</code>	Adicionados <code>setDataValues()</code> e <code>_computeStatsFromLoadedData()</code>
<code>source/tools/FitterDummyImpl.h/.cpp</code>	Adicionado stub vazio
<code>source/tools/DataAnalyzer_if.h</code>	Novo (interface + structs de saída)
<code>source/tools/DataAnalyzerDefaultImpl1.h</code>	Novo (declaração da classe)
<code>source/tools/DataAnalyzerDefaultImpl1.cpp</code>	Novo (≈ 810 linhas, toda a lógica)
<code>source/tools/TraitsTools.h</code>	Adicionada especialização para <code>DataAnalyzer_if</code>
<code>source/tools/CMakeLists.txt</code>	Adicionado <code>DataAnalyzerDefaultImpl1.cpp</code>
<code>source/tools/main.cpp</code>	Adicionado comando <code>--demo</code>

2.2 Extensão da interface `Fitter_if` (M1)

`FitterDefaultImpl` só aceitava dados via arquivo binário (`setDataFilename`). Para que o `DataAnalyzer` pudesse repassar ao fitter um vetor já carregado em memória, foi adicionado o método `setDataValues(const std::vector<double>&)` à interface `Fitter_if` e à implementação concreta.

A lógica interna de `FitterDefaultImpl` foi refatorada: o código que computava estatísticas de cache foi extraído para o método privado `_computeStatsFromLoadedData()`, chamado tanto pelo caminho original de arquivo binário quanto pelo novo caminho de memória. Os flags de cache `_cacheLoaded` e `_cacheUsable` são definidos por ambos os caminhos, de modo que o mecanismo de *lazy loading* existente em `_ensureDataLoaded()` enxerga os dados como já carregados e não tenta abrir arquivo algum.

2.3 Interface `DataAnalyzer_if` e structs de saída (M2)

A interface `DataAnalyzer_if` define cinco structs de saída puramente de dados (sem métodos):

- `SummaryStatistics` – `n`, mín, máx, amplitude, média, mediana, moda, `Q1`, `Q3`, variância, desvio-padrão, `CV`, assimetria, curtose;
- `HistogramData` – número de classes, limites inferiores, frequências absolutas e relativas;
- `BoxplotData` – mín, `Q1`, mediana, `Q3`, máx, `IIQ`, lista de *outliers*;
- `FitResult` – nome da distribuição, `SSE`, até cinco parâmetros, flag de validade;
- `GoFResult` – nome, estatística de teste, `p`-valor, valor crítico, nível de significância, decisão, conclusão textual.

A grafia americana (*Analyzer*) foi adotada para evitar confusão com o antigo `DataAnalyser_if` (grafia britânica), que possui dependência de `ExperimentManager_if` e pertence a outra camada arquitetural.

2.4 Implementação: carregamento de dados e estatísticas descritivas (M3)

O carregamento de dados suporta dois caminhos:

- `setDataValues(vector<double>)`: armazena os dados em memória, ordena o vetor em `_sortedData` e computa média, variância e desvio-padrão em uma única passagem (`_rebuildCache()`);
- `setDataFilename(string)`: utiliza `SimulationResultsDatasetParser::loadFromTextFile` para suportar os formatos de texto do GenESyS (`RawNumeric`, `RecordLegacy`, `RecordEnriched`, `GuiTabular`), depois chama `setDataValues`.

Os quantis são calculados pelo método R7 (interpolação linear): dado percentil p , calcula-se $h = (n - 1)p$, $i = \lfloor h \rfloor$, $f = h - i$, e o resultado é $x_{(i)}(1 - f) + x_{(i+1)}f$. Assimetria e curtose são calculadas diretamente como momentos padronizados de terceira e quarta ordem (curtose em excesso, subtraindo 3).

Decisão de projeto – `StatisticsDataFile_if`: A proposta sugeria encaminhar as estatísticas descritivas pela interface `StatisticsDatafile_if`. Essa interface estende `CollectorDatafile_if`, que é estruturalmente acoplada a um arquivo binário de coletor – não há construtor nem setter que aceite `vector<double>`. Utilizá-la exigiria escrever os dados em arquivo binário e relê-los, o que derrota o propósito de operação em memória. A decisão foi calcular as estatísticas diretamente, produzindo resultados equivalentes sem o custo de I/O.

2.5 Ajuste de distribuições (M4)

O método `fitDistribution(name)` despacha para o método correto de `FitterDefaultImpl` com base no nome da distribuição e empacota SSE e parâmetros em `FitResult`. As sete distribuições suportadas e seus parâmetros são:

`uniform` (mín, máx), `triangular` (mín, moda, máx), `normal` (média, dp), `exponential` (média), `erlang` (média, m), `beta` (α , β , inf, sup), `weibull` (forma, escala).

O método `fitAll()` chama `_fitter.fitAll()` para obter o nome da melhor distribuição e, em seguida, chama `fitDistribution(nome)` para recuperar os parâmetros. O duplo ajuste é intencional: `fitAll` retorna apenas nome e SSE.

2.6 Testes de aderência (M5 e M6)

Qui-quadrado (M5): O número de classes é determinado pela fórmula de Sturges ($k = \max(5, 1 + \lceil \log_2 n \rceil)$). Classes com frequência esperada $E < 5$ são fundidas com a adjacente. O grau de liberdade é $k_{\text{fundido}} - 1 - p$, onde p é o número de parâmetros estimados a partir dos dados. O p-valor é calculado integrando numericamente a PDF da distribuição qui-quadrado usando `SolverDefaultImpl1`; o valor crítico é obtido por bissecção sobre a CDF acumulada.

Kolmogorov-Smirnov (M6): A estatística $D_n = \max_i |F_n(x_i) - F(x_i)|$ é calculada sobre as estatísticas de ordem. O p-valor usa a série assintótica de Kolmogorov com 200 termos ($2 \sum_{k=1}^{200} (-1)^{k+1} e^{-2k^2(\sqrt{n} D_n)^2}$); o valor crítico usa a fórmula aproximada $D_\alpha \approx \sqrt{-\ln(\alpha/2)/(2n)}$. A aproximação assintótica subestima levemente o p-valor para $n < 35$.

2.7 Análise de séries temporais (M7)

- `movingAverage(w)`: janela deslizante com soma acumulada $O(n)$; saída com $n - w + 1$ elementos;
- `autocorrelation(maxLag)`: estimador enviesado $\hat{r}(k) = \hat{c}(k)/\hat{c}(0)$, onde $\hat{c}(k) = \frac{1}{n} \sum_{i=0}^{n-k-1} (x_i - \bar{x})(x_{i+k} - \bar{x})$; `maxLag` é silenciosamente limitado a $n - 1$;
- `correlogram(maxLag)`: retorna o mesmo cálculo de `autocorrelation`, diferenciado conceitualmente para uso como gráfico de barras por defasagem.

2.8 Inferência paramétrica (M8)

Os métodos de intervalo de confiança e teste de hipóteses delegam para `HypothesisTesterDefaultImpl` usando as estatísticas em cache (`_sampleMean`, `_sampleVariance`, `_sampleStddev`, `_count`, `_confidenceLevel`). Os testes para duas populações requerem que `loadSecondSample()` tenha sido chamado previamente.

Correção de bug: A versão inicial de `testProportionTwoSamples` e `testVarianceTwoSamples` invocava, por engano, as sobrecargas de uma população de `HypothesisTesterDefaultImpl1`, tratando a estatística da segunda amostra como valor da hipótese nula. A correção foi passar `_count2` como quarto argumento, selecionando as sobrecargas corretas de duas populações já existentes na implementação.

2.9 Integração com o GenESyS

O componente integra-se ao simulador por meio do mecanismo `TraitsTools<T>`:

```
1 template <> struct TraitsTools<DataAnalyzer_if> {  
2     typedef DataAnalyzerDefaultImpl1 Implementation;  
3 };
```

Listing 1: Registro da implementação em `TraitsTools.h`

Qualquer código do simulador pode obter uma instância concreta por meio de `TraitsTools<DataAnalyzer_if>::Implementation`, sem depender da classe concreta diretamente. A ferramenta de linha de comando (`genesys_tools_application`) foi estendida com o comando `--demo` para demonstrar e validar o componente sem interface gráfica.

3 Validação

3.1 Estratégia de teste

A validação foi realizada por meio de um *harness* de testes automatizados integrado ao executável `genesys_tools_application`, acionado pelo comando `--demo`. O harness instancia `DataAnalyzerDefaultImpl1` diretamente e executa 26 verificações organizadas em 7 seções, imprimindo `[PASS]` ou `[FAIL]` para cada uma.

Três conjuntos de dados fixos são usados:

- `kNormalData`: 50 valores amostrados de $N(5, 1)$;
- `kExpoData`: 20 valores amostrados de $\text{Exp}(2)$ (média = 2);
- `kTemporalData`: 20 valores com tendência crescente, de 1,1 a 3,0.

3.2 Seção 1 – Carregamento de dados

- `setDataValues` aceita 50 valores e `summaryStatistics().n == 50`;
- `setDataFilename` carrega os mesmos 50 valores de arquivo `RawNumeric` temporário;
- `clearData` esvazie o conjunto (`n == 0`);
- `loadSecondSample` armazena um segundo vetor de 20 valores.

3.3 Seção 2 – Estatísticas descritivas

- Média dentro do intervalo $[4,5; 5,5]$ para dados $N(5, 1)$;
- Ordenamento: $\text{mín} < Q_1 < \text{mediana} < Q_3 < \text{máx}$;
- Desvio-padrão > 0 ;
- Consistência entre `quartile(2)`, `decile(5)` e `centile(50)` (todos devem ser iguais à mediana).

3.4 Seção 3 – Histograma e boxplot

- Soma das frequências absolutas igual a $n = 50$;
- Histograma com exatamente 8 classes;
- Soma das frequências relativas igual a 1,0 (tolerância 10^{-10});
- $IIQ = Q_3 - Q_1$ (verificação da fórmula do boxplot).

3.5 Seção 4 – Ajuste de distribuições

- Ajuste normal válido (`FitResult.valid == true`);
- Parâmetro de média do ajuste normal dentro de $[4,5; 5,5]$;
- `fitAll` retorna resultado válido;

- SSE do `fitAll` $\leq$ SSE do ajuste normal (é a melhor entre todas);
- Média do ajuste exponencial nos dados `kExpoData` dentro de $[1,0; 3,5]$ (esperada ≈ 2).

3.6 Seção 5 – Testes de aderência

- Teste qui-quadrado não rejeita ajuste normal a $\alpha = 0,05$ (dados são normais);
- Teste KS não rejeita ajuste normal a $\alpha = 0,05$.

3.7 Seção 6 – Séries temporais

- `movingAverage(3)` produz $n - w + 1 = 18$ valores;
- Primeiro valor da média móvel igual à média dos três primeiros elementos $((1,1 + 1,3 + 1,2)/3)$;
- `autocorrelation(5)` retorna `maxLag + 1 = 6` valores;
- $\hat{r}(0) = 1,0$ por construção;
- $\hat{r}(1) > 0,5$ para dados com tendência crescente;
- `correlogram(5) == autocorrelation(5)` (implementação compartilhada).

3.8 Seção 7 – Inferência paramétrica

- IC 95% para a média contém o verdadeiro valor 5,0;
- Teste t bilateral para $\mu_0 = 5,2$ não rejeita (μ_0 próximo à média amostral de 5,206);
- Teste t bilateral para $\mu_0 = 10,0$ rejeita (valor muito afastado);
- Limite superior do IC de variância $>$ limite inferior;
- Teste de médias para duas populações (normal vs. exponencial) rejeita $H_0 : \mu_1 = \mu_2$.

3.9 Resultado da execução

A saída do comando `--demo` é apresentada abaixo (trecho):

```
=====
DataAnalyzer Demo - GenESyS tools
=====
...
1. DATA LOADING
-----
[PASS] setDataValues accepted 50 values
[PASS] setDataFilename loaded same 50 values from RawNumeric text file
[PASS] clearData empties the dataset
[PASS] loadSecondSample stores a second vector
...
```

=====

Demo complete.

=====

Todos os 26 testes passam com a implementação atual.

3.10 Problemas encontrados e corrigidos

Durante o desenvolvimento foram encontrados e corrigidos três problemas:

1. **Incompatibilidade de tipos no CDF de Erlang:** `std::llround` retorna `long long int`, enquanto a literal `1L` é `long int`. O compilador recusou `std::max(1L, std::llround(p2))` por tipos conflitantes. Corrigido com `std::max(1LL, std::llround(p2))`.
2. **Métodos não-const de ConfidenceInterval:** Os métodos `inferiorLimit()` e `superiorLimit()` de `HypothesisTester_if::ConfidenceInterval` não são declarados `const`, portanto variáveis locais `const auto ci` falham com erro de qualificador. Corrigido declarando as variáveis `CI` como `auto ci` (não-const).
3. **Testes de duas populações usando sobrecargas de uma população:** As versões iniciais de `testProportionTwoSamples` e `testVarianceTwoSamples` omitiam o argumento `_count2`, ativando as sobrecargas de uma população e tratando a estatística da segunda amostra como valor nulo. Corrigido passando `_count2` como quarto argumento, selecionando as sobrecargas de duas populações corretas.

4 Conclusão

Este trabalho implementou o componente `DataAnalyzer` para o GenESyS, atendendo à diretriz 11.3.1 (*foco maior em diagnósticos adicionais e ajustes de curvas*). O componente está efetivamente integrado ao simulador por meio do padrão `TraitsTools<DataAnalyzer_if>` e expõe as seguintes capacidades:

1. **Carregamento de dados** – a partir de vetores em memória ou de arquivos texto nos quatro formatos suportados pelo GenESyS;
2. **Estatísticas descritivas** – n, mínimo, máximo, amplitude, média, mediana, moda, Q1, Q3, variância, desvio-padrão, CV, assimetria e curtose;
3. **Quantis** – acessores diretos para quartis, decis e centis (método R7);
4. **Histograma** – número de classes configurável, limites, frequências absolutas e relativas;
5. **Boxplot** – Q1, mediana, Q3, IIQ, limites de whisker e identificação de outliers;
6. **Ajuste de distribuições** – sete distribuições com seleção automática da melhor por SSE;
7. **Testes de aderência** – qui-quadrado (com fusão de classes $E < 5$) e Kolmogorov-Smirnov;
8. **Análise de séries temporais** – média móvel, autocorrelação e correlograma;
9. **Inferência paramétrica** – intervalos de confiança e testes de hipóteses para média, proporção e variância, em uma e duas populações.

O desenvolvimento exigiu a extensão da interface `Fitter_if` com o método `setDataValues`, a criação de uma nova interface pública `DataAnalyzer_if` com cinco structs de saída, a implementação de rotinas numéricas para CDF e quantis das distribuições suportadas reutilizando `SolverDefaultImpl1`, e a correção de um bug nas sobrecargas de testes de duas populações.

A validação por 26 testes automatizados confirma o funcionamento correto de todas as funcionalidades. O componente está pronto para ser consumido por outros módulos do GenESyS, incluindo futuros painéis gráficos de análise de resultados de simulação.

Referências

- [1] Averill M. Law. *Simulation Modeling and Analysis*. McGraw-Hill, 5 edition, 2015.
- [2] Renato Rego Leal Cancian. GenESyS – generic extensible simulator. <https://github.com/rlcancian/Genesys-Simulator>, 2024. Branch 2026-1.